

02 — Frontend Architecture

Product: Endpoint IQ **Document:** Frontend Architecture (React + TypeScript)

1. Application Architecture

Endpoint IQ's frontend is a **single-page application (SPA)** built with React and TypeScript. It talks exclusively to the **API Gateway** over HTTPS (REST for CRUD/config actions, WebSocket for live test-run progress, per the architecture's REST / WS link between Frontend and API Gateway).

```
flowchart TB
    subgraph Browser
        FE[React/TypeScript SPA]
    end
    FE -- "HTTPS REST" --> GW[API Gateway]
    FE -- "WS (test run progress)" --> GW
    GW --> AUTH[Auth Service]
    GW --> SPEC[Spec Import Service]
    GW --> ORCH[Orchestrator]
    GW --> AI[AI Analysis Engine]
    GW --> REPORT[Report Service]
```

The frontend never talks directly to the target API under test, the Job Queue, Workers, the AI Engine, or the database. All of that is mediated by the API Gateway, consistent with the architecture's "single front door" description of the Gateway (§4.2).

2. Feature-Based Structure

The frontend is organized **by feature/workflow-step**, not by generic technical layer, because Endpoint IQ's UI is a fairly linear multi-stage workflow (project → import → endpoints → roles/credentials → run → results → AI analysis → report). A feature-based structure keeps everything needed to build or modify one workflow stage in one place, which is easier for both human developers and AI coding agents to reason about in isolation.

```
frontend/
├── src/
│   ├── app/           # App shell: providers, router setup, global layout wiring
│   ├── components/    # Shared, generic UI components (Button, Table, Modal, Badge, etc.)
│   └── features/
│       ├── projects/  # Project CRUD (create/list/view)
│       ├── spec-import/ # Swagger/OpenAPI import flow
│       ├── endpoints/  # Endpoint discovery/viewer
│       ├── roles/      # Role configuration
│       ├── credentials/ # Credential configuration
│       ├── test-runs/   # Start run, progress monitoring
│       ├── test-results/ # Results dashboard, failed API detail
│       ├── ai-analysis/ # AI analysis / recommendations UI
│       └── reports/     # Report view/download
│   ├── pages/         # Route-level components that compose feature components
│   ├── layouts/        # AppLayout (sidebar+header), AuthLayout, etc.
│   ├── services/       # API client modules (one per backend domain)
│   └── hooks/          # Shared cross-feature hooks (e.g. usePolling, useWebSocket)
```

```

|   | types/           # Shared TypeScript types/interfaces (see 06-...)
|   | utils/          # Formatting, validation, constants
|   | stores/         # Global UI/client state (see State Management below)
|   | routes/         # Route definitions/config

```

Each features/<name>/ folder internally follows:

```

features/test-results/
| components/        # Feature-local components (ResultsTable, EndpointResultRow...)
| hooks/            # Feature-local hooks (useTestResults, useFailedEndpointDetail...)
| api.ts            # Feature-local API calls (thin wrapper over services/)
| types.ts          # Feature-local types not shared elsewhere
| index.ts

```

Decision: The prompt's suggested generic structure (components/, features/, pages/, etc.) is kept largely as-is because it already fits Endpoint IQ well — the product's real complexity lives in per-workflow-step logic (import parsing display, run monitoring, AI analysis rendering), which maps cleanly to features/. No restructuring was necessary beyond nesting components/hooks/api/types inside each feature folder for locality.

3. Component Architecture

Three tiers:

1. **Primitive/shared components** (components/) — Button, Input, Select, Table, Modal, Toast, Tabs, Badge, StatusPill, CodeBlock, JsonViewer. No knowledge of Endpoint IQ domain concepts.
2. **Feature components** (features/*/components/) — domain-aware, e.g. EndpointList, RoleForm, TestRunProgressBar, FailedEndpointDetail, AIAalysisCard. Compose primitives.
3. **Page components** (pages/) — route targets that fetch data (via hooks) and lay out feature components. Pages contain minimal logic; they orchestrate, not implement.

4. Routing Architecture

React Router (or equivalent) with routes nested under an authenticated AppLayout. See **04-Frontend-Pages-and-Routes.md** for the full route table. Routes are project-scoped (/projects/:projectId/...) reflecting the data model where roles, endpoints, credentials, test_runs, and reports all belong to a project.

5. API Communication Layer

A dedicated services/ layer wraps all HTTP/WS calls:

- One service module per backend domain: projectService, specImportService, endpointService, roleService, credentialService, testRunService, resultService, aiAnalysisService, reportService.
- A single shared HTTP client instance (e.g. Axios or fetch wrapper) configured with base URL, interceptors, and error normalization — see **05-Frontend-API-Integration.md**.
- A WebSocket (or polling fallback) manager for live test-run progress, isolated in features/test-runs/.

6. Authentication Handling

The architecture defines JWT/OIDC authentication and HttpOnly cookie storage **for the target API under test** (§4.3, §6). It does **not** explicitly define how a human user authenticates *into the Endpoint IQ platform itself**.

Open Decision: Platform-level user authentication (login page, session handling for the Endpoint IQ user, as opposed to the target API's role credentials) is not defined in the source document. The frontend architecture assumes a /login page exists and that the API Gateway issues/validates a platform-session cookie (HttpOnly, consistent with the project's existing "never in localStorage" security stance), but the concrete mechanism (JWT vs. session vs. SSO/OIDC) must be confirmed with the backend team. See 04-Frontend-Pages-and-Routes.md and 05-Frontend-API-Integration.md for how this is flagged.

Regardless of mechanism, the frontend rule is fixed by the architecture's security section: **no auth token of any kind is stored in localStorage/sessionStorage**; the browser relies on HttpOnly cookies set by the backend, and the frontend only reads non-sensitive session metadata (e.g. "logged in as X") if the backend exposes a /me-style endpoint.

7. State Management

Server state vs. UI state

State type	Examples	Where it lives
Server state (data owned by the backend, fetched/cached/invalidated)	Projects, endpoints, roles, credentials (metadata only), test runs, test results, AI analysis, reports	A server-state/data-fetching library (e.g. React Query/TanStack Query) — cache keyed by resource, revalidated on mutation
UI state (ephemeral, client-only)	Active tab, modal open/closed, table filters/sort, form draft values, sidebar collapsed	Local component state (useState) or a lightweight global UI store (stores/) only when shared across distant components (e.g. global toast queue)
Real-time/streaming state	Live test-run progress (jobs completed/total, currently-running endpoint)	Dedicated hook (useTestRunProgress) backed by WebSocket/polling, exposed as server-state-like data

Rule of thumb: if it can be refetched from the backend and be correct, it's server state and belongs in the data-fetching cache, not in a global store. Global stores (stores/) are reserved for cross-cutting client-only concerns (auth/session flag, theme, toast notifications).

8. Error Handling

- Every API service call funnels errors through a shared normalizer that classifies: network error, timeout, 401/403 (auth), 400/422 (validation), 5xx (server), and target-API-specific errors surfaced during test runs (these are *data*, not frontend errors — see 07-...).
- Page-level error boundaries catch rendering errors; feature-level try/catch + toast for action errors (e.g. "Failed to start test run").

- See **05-Frontend-API-Integration.md** for the full error taxonomy and **08-Frontend-Development-Guidelines.md** for conventions.

9. Loading Handling

- Skeleton loaders for initial page/data loads (endpoint list, results table).
- Inline spinners/disabled states for actions (Start Test Run button while request is in-flight).
- Progressive loading for the test-run monitor: show the run shell immediately, stream progress in as it arrives.

10. Reusable Components

Minimum shared component set (detailed visually in 03-UI-UX-Specification.md): Button, Input, Select, Checkbox, Table, Tabs, Modal, Toast, Badge/StatusPill, CodeBlock, Json-Viewer, RequestResponseViewer, EmptyState, ErrorState, Skeleton, ConfirmDialog, ProgressBar.

11. Shared Utilities

utils/ includes: date/duration formatting, HTTP status code → label/color mapping, response diffing helper (for expected vs. actual), file download helper (for report downloads), form validators, environment/config accessor.

12. TypeScript Types

Central domain types live in types/ and mirror the backend data model 1:1 (Project, Endpoint, Role, Credential, TestRun, TestResult, AIAnalysis, Report) — see **06-Frontend-State-and-Data-Models.md** for full definitions. Feature-local types (form state, view-model shapes) stay inside the feature folder.

13. Environment Configuration

Variable	Purpose	Notes
VITE_API_BASE_URL (or equivalent)	Base URL of the API Gateway	Required
VITE_WS_BASE_URL	WebSocket endpoint base for live progress	Required if WS is confirmed (see Open Decision in 05-...)
VITE_ENV	development / staging / production	Controls logging verbosity, etc.

Open Decision: Exact env var names/build tool (Vite vs. CRA vs. Next.js) are not specified in the source architecture, which only mandates “React / TypeScript.” This document assumes a Vite-based SPA (fast, standard for TS SPAs) unless the team specifies otherwise. See 08-Frontend-Development-Guidelines.md.

Cross-references: 04-Frontend-Pages-and-Routes.md (routes), 05-Frontend-API-Integration.md (service layer detail), 06-Frontend-State-and-Data-Models.md (types).